

Econometrics Student Experiences the Machine Learning Framing of Regression, For Once in His Life

Kavi Mookherjee Amodt

July 1, 2026

Abstract

Here I will do two things: (A) I will explain the machine learning framing of regression to someone who knows regression only as it is framed in econometrics (or in social sciences more generally), and (B), I will show how I implemented linear regression, logistic regression, and normalization entirely from scratch in Python. I write almost entirely in bullet points because that's how I like to learn. Each bullet deepens understanding; all of them can be seen as having approximately equal semantic weight, so that you don't have to decide whether what you are reading is fluff or not; and you can pace yourself metronomically, it being easy to return to your last checkpoint of understanding.

Part (A): Regression in ML and in the Social Sciences

1 Inference vs prediction in statistics

1.1 In econometrics we infer

- In econometrics, regression is the engine of **inference**.
- Inference, in this sense, means we use existing data to infer the *relationship* between some variables (e.g. x = income and y = SAT score).
- To be more precise, this relationship is a statistical distribution. Our goal is to model that distribution.
- When we do regression, we assume that there's a *true* distribution; a **data generating process**.
- When we do linear regression, we assume that the true data generating process looks like a (multivariate) linear equation:

$$y = X\beta + \epsilon$$

- Our **object of interest** is the vector of weights: β .
- Put differently, β is the vector of true relationships between our variables. ϵ is the noise (more on this later).
- Our goal is to create an estimate of the coefficient vector β —that estimate is denoted $\hat{\beta}$ —which comes closest, when multiplied by the **design matrix** X , to the data for y :

$$\hat{y} = X\hat{\beta}$$

1.2 In ML we predict

- In machine learning, our primary goal is not to obtain and report a relationship (like $\hat{\beta}$), but rather to use data from the world to *make more data*.
- In other words, we take a complex set of causal relationships out there in the world—a set of samples from a data generating process—and use it to **predict** what that process would output.
- An LLM doesn't explicitly mimick the causal process in your brain which leads to language (emotional disregulation, excitement, boredom, etc); it finds complex statistical *relationships* between tokens (samples from the process) and predicts what the causal process in your brain would output (the next word).
- Thus, our object of interest is not β , but y .
- Instead of focusing on obtaining and reporting $\hat{\beta}$, we try to make a prediction: $\hat{y}$.

1.3 Throw ϵ out the window (for now)

- FINISH THIS

2 Linear regression is just projecting onto $\text{span}(\mathbf{X})$

2.1 Gradient of the squared-error loss function

- Notice how, in the ML framing, there is no notion of probability. It's just pure optimization of this loss function:

$$L(\beta) = \frac{1}{n} \|y - X\beta\|^2$$

- Remember that $\|v\| = v^T v$ and $(A - B)^T = A^T - B^T$
- Therefore, in matrix form:

$$L(\beta) = (y - X\beta)^T (y - X\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta$$

- Thus, setting $\nabla L = 0 \implies -2X^T y + 2X^T X \beta = 0 \implies \hat{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
- This result is called the **normal** equation; we use this to compute our estimate of the relationship vector $\hat{\beta}$.
- Crucially, this is the *closed-form* way to compute $\hat{\beta}$ for linear regression—we don't have such a luxury in logistic regression.

2.2 H, the Hat matrix

- Plugging our normal equation for $\hat{\beta}$ back into the equation for our $\hat{y}$ prediction:

$$\hat{y} = X\hat{\beta} = X(X^T X)^{-1} X^T y = \mathbf{H}y$$

- $\mathbf{H}$ is the Hat matrix of X : multiplying a vector y by the Hat matrix of X gives you the **orthogonal projection** of y onto X .

- That’s all OLS is: our estimator for y is the point in $\text{span}(X)$ which is closest, in terms of Euclidean distance, to y .
- That mechanic right there—finding the closest point in representable space to our data-point—is one you will see throughout linear-algebraic machine learning: in QR decomposition, principal components analysis, support vector machines.

3 Gradient descent for OLS

3.1 Why use gradient descent?

- If we have a closed-form way to compute our coefficients, why is gradient descent considered the *sledgehammer* of modern ML?
- Unsurprisingly, the answer is efficiency: computing $X^T X$ is $O(p^3)$ where p is the number of predictors.
- We thus use GD in both ML and econometric settings.

3.2 How GD works

- The gradient of a continuous, multivariate function (in this case, our loss function L) is a vector which points in the direction of **steepest ascent**.
- As we saw earlier, it is denoted ∇L . Its magnitude $\|\nabla L\|$ equals the slope of L in that direction.
- GD is a process:
 1. Find ∇L at the point on the “surface” of the function L where you stand now.
 2. The vector $\hat{\beta} \leftarrow \hat{\beta} - \mu \nabla L$.
 3. Travel to the t

4 OLS is MLE under Gaussian noise

5 Logistic regression doesn’t have closed form

6 Regularizing is biasing on purpose

Part (B): Implementation Details